



TypeScript, NPM, and Vite

Part 2

Vite, Build Tooling & Project Architecture

Chapter Overview

- 01** Vite — Modern Build Tooling
- 02** Vite Configuration & Environment
- 03** Asset Handling in Vite
- 04** Project Architecture
- 05** TypeScript Best Practices
- 06** Preparing for React



Vite — Modern Build Tooling

Section 1



Why Vite?

The Build Tool Problem

- Webpack popularized bundling — but dev startup became painfully slow on large projects
- As projects grow, Webpack dev server takes 30-60+ seconds to start

Vite's Approach

- Dev mode: no full app bundling — serves native ES modules (dependencies pre-bundled with esbuild)
- Build mode: uses Rollup for optimized production bundles
- Hot Module Replacement (HMR): changes reflect instantly, not after a full rebuild

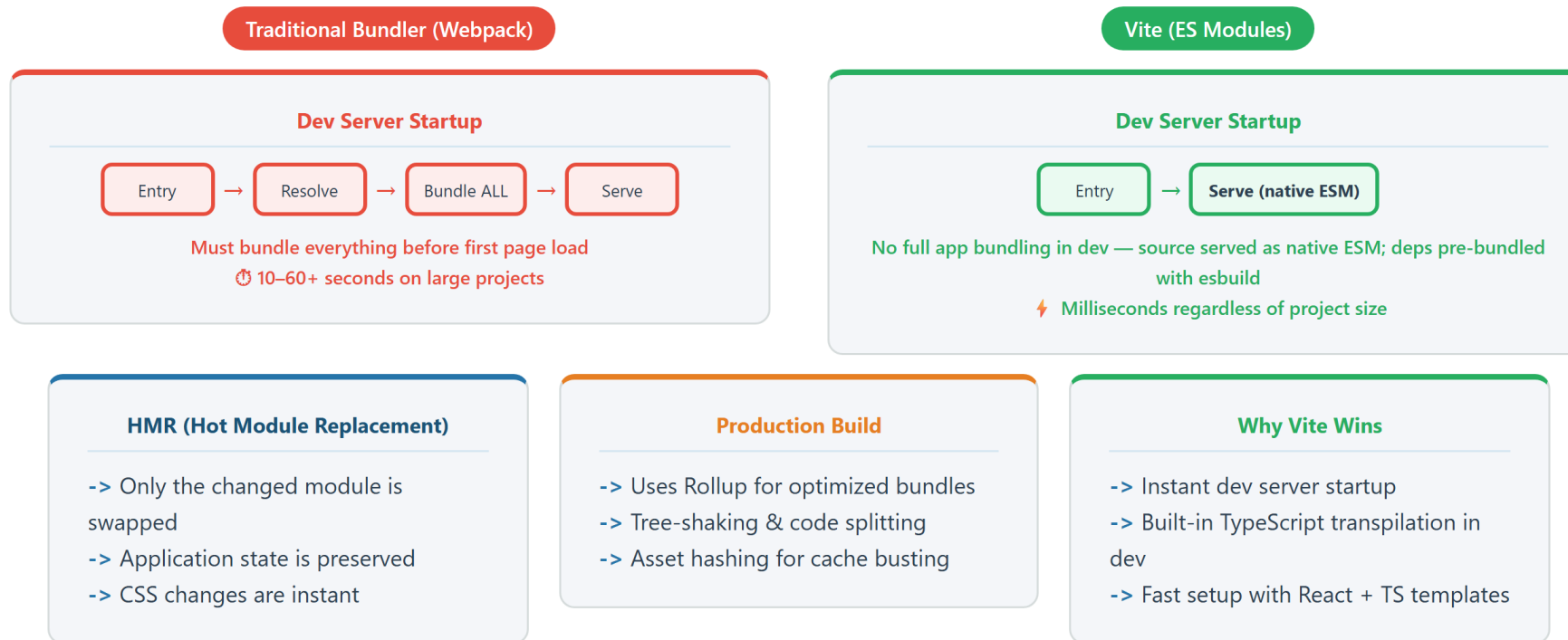
Result

- Dev server starts in milliseconds regardless of project size
- Changes appear in the browser in under 50ms



Vite vs Traditional Bundlers

Vite vs Traditional Bundlers



How Vite's Dev Server Works

Dependency Pre-Bundling

- On first run, Vite scans imports and pre-bundles node_modules with esbuild (10-100x faster than Webpack)
- Results are cached in node_modules/.vite — subsequent starts are instant

Native ES Module Serving

- Your source files are served as-is via native browser ESM imports
- The browser requests modules on demand — no full bundle needed
- TypeScript files are transpiled on-the-fly by esbuild (no type checking in dev)

Important Distinction

- Vite transpiles TS but does NOT type-check — run tsc --noEmit separately or via IDE
- This is why dev server is fast: it skips the expensive type-checking step



Creating a Vite Project

Scaffolding

- `npm create vite@latest my-app -- --template react-ts`
- Templates: vanilla-ts, react-ts, vue-ts, svelte-ts

What You Get

- Pre-configured TypeScript, React, and dev server
- `index.html` as the entry point (not a JavaScript file)
- `src/` directory with `App.tsx` and `main.tsx`

First Steps

- `cd my-app → npm install → npm run dev`
- Browser opens at `localhost:5173` (default port, commonly changed to 3000)



Scaffold and Run

```
# Create a new React + TypeScript project
npm create vite@latest my-app -- --template react-ts

# Install dependencies
cd my-app
npm install

# Start dev server
npm run dev
# → Local: http://localhost:5173/

# Build for production
npm run build
# → Output in dist/

# Preview production build locally
npm run preview
```



Hot Module Replacement (HMR)

What Is HMR?

- When you save a file, only the changed module is replaced — not the entire page
- Application state (form inputs, scroll position) is preserved

How Vite HMR Works

- Vite watches files via the file system
- On change, it sends the updated module over WebSocket
- The browser swaps just that module — no full reload
- CSS changes are instant — injected without any page update at all



Vite Configuration & Environment

Section 2



vite.config.ts — Core Configuration

Configuration File

- TypeScript configuration file in the project root
- Uses defineConfig() for type-safe configuration with autocomplete
- Plugins: @vitejs/plugin-react for React Fast Refresh support

Common Options

- server.port — change the dev server port (default 5173)
- server.open — automatically open browser on start
- build.outDir — where production files go (default 'dist')
- resolve.alias — path shortcuts like @ → ./src



vite.config.ts

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import path from "path";

export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000,
    open: true,
  },
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "./src"),
    },
  },
});
```

Environment Variables

How Vite Handles Environment Variables

- Variables must be prefixed with `VITE_` to be exposed to client code
- Access via `import.meta.env.VITE_API_URL` (not `process.env`)
- Variables without `VITE_` prefix are only available in `vite.config.ts` (server-side)

.env File Hierarchy

- `.env` — loaded in all environments
- `.env.local` — local overrides (gitignored)
- `.env.development` — loaded only during `npm run dev`
- `.env.production` — loaded only during `npm run build`
- Mode-specific files take priority (`.env.development` overrides `.env`)



.env Files in Practice

```
# .env – shared across all environments
VITE_APP_TITLE=My Application

# .env.development – dev only
VITE_API_URL=http://localhost:8080/api

# .env.production – production only
VITE_API_URL=https://api.example.com

# Accessing in TypeScript:
const apiUrl = import.meta.env.VITE_API_URL;
const title  = import.meta.env.VITE_APP_TITLE;
const isDev  = import.meta.env.DEV; // built-in boolean
const mode   = import.meta.env.MODE; // "development" | "production"
```



Path Aliases and Absolute Imports

The Problem with Relative Imports

- Deep nesting leads to fragile paths: ../../../utils/format
- Moving a file breaks all its relative imports

Path Aliases

- Configure @ to point to src/ — then import from @/utils/format
- Needs configuration in both vite.config.ts AND tsconfig.json
- Vite resolves the alias at build time; TypeScript needs it for type checking



Path Alias Configuration

```
// vite.config.ts – Vite needs this to resolve imports
resolve: {
  alias: { "@": path.resolve(__dirname, "./src") }
}

// tsconfig.json – TypeScript needs this for type checking
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": { "@/*": ["src/*"] }
  }
}

// Now use clean imports anywhere:
import { formatDate } from "@/utils/format";
import type { User } from "@/types/user";
import { Button } from "@/components/Button";
```



Asset Handling in Vite

Section 3



CSS in Vite

Plain CSS

- Import CSS directly in TypeScript: `import './App.css'` — Vite injects it automatically
- HMR works on CSS files — changes appear instantly without page reload

CSS Modules

- Name your file `.module.css` — Vite scopes class names automatically
- import styles from `'./Button.module.css'` — `styles.primary` becomes a unique class name
- Prevents style collisions between components

CSS Preprocessors

- Install sass, less, or stylus — Vite detects and uses them automatically
- No configuration needed: `npm install -D sass`, then use `.scss` files



CSS Modules Example

```
/* Button.module.css */
.primary {
  background: #7500C0;
  color: white;
  padding: 8px 16px;
  border-radius: 4px;
}
.secondary {
  background: transparent;
  border: 1px solid #7500C0;
  color: #7500C0;
}

// Button.tsx
import styles from "./Button.module.css";
// styles.primary → "Button_primary_x7d2k" (auto-scoped)
const Button = () => <button className={styles.primary}>Click</button>;
```



Static Assets and the Public Directory

Importing Assets

- import logo from './logo.svg' — returns a URL string for use in src attributes
- Assets under 4KB are inlined as base64 (configurable via build.assetsInlineLimit)
- Larger assets get content-based hash filenames for cache busting

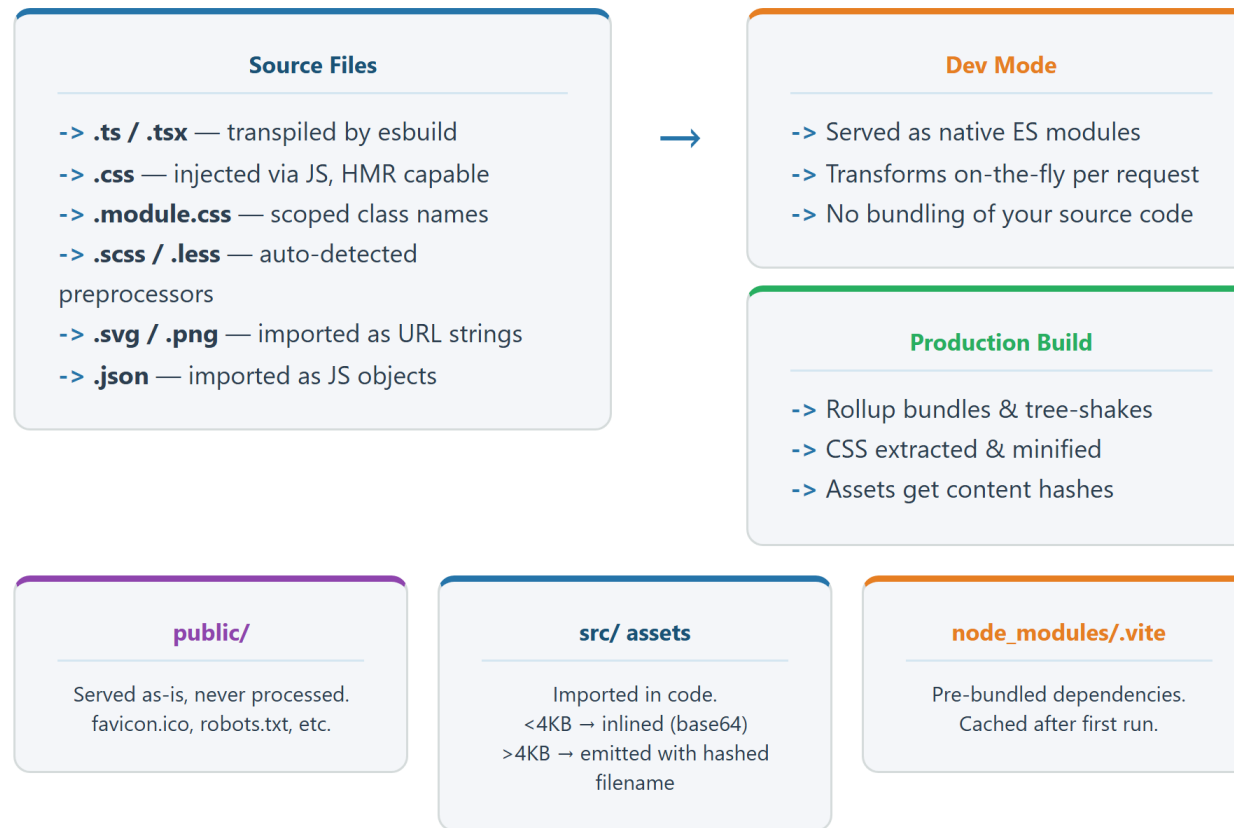
The public/ Directory

- Files in public/ are served at root path as-is — never processed by Vite
- Use for favicon.ico, robots.txt, and assets that need fixed URLs
- Reference with absolute paths: /favicon.ico (no import needed)



Vite Asset Pipeline

How Vite Handles Assets



Project Architecture

Section 4



Project Structure Best Practices

Directory Layout

- src/ — all source code lives here
- src/components/ — reusable UI components
- src/types/ — shared TypeScript interfaces and types
- src/utils/ — helper functions
- src/hooks/ — custom React hooks (in React projects)

File Naming

- .ts for pure TypeScript, .tsx for files with JSX
- PascalCase for components: UserCard.tsx
- camelCase for utilities: formatDate.ts



Project Structure

Vite + React + TypeScript Project Structure

Project Root

```
my-app/
├── index.html      ← Vite entry point
├── package.json    ← deps & scripts
├── tsconfig.json   ← TS configuration
├── vite.config.ts  ← Vite plugins & options
├── src/
│   ├── main.tsx    ← App bootstrap
│   ├── App.tsx     ← Root component
│   ├── components/  ← Reusable UI
│   ├── hooks/       ← Custom hooks
│   ├── types/       ← Shared TS types
│   ├── utils/       ← Helper functions
│   └── App.css      ← App-level styles
├── dist/           ← Production build output
└── node_modules/   ← Installed packages
```

File Extensions

.tsx Components (with JSX) **.ts** Pure TypeScript

Naming Conventions

- > **PascalCase** — components: `UserCard.tsx`
- > **camelCase** — utilities: `formatDate.ts`
- > **kebab-case** — CSS: `user-card.css`

Key Config Files

- > **tsconfig.json** — strict mode, module resolution
- > **vite.config.ts** — plugins, aliases, server port
- > **eslint.config.js** — ESLint flat config
- > **.env** — environment variables (VITE_ prefix)



Build and Deploy

Production Build

- npm run build — TypeScript compiles, Vite bundles and optimizes
- Output goes to dist/ — static HTML, CSS, and JS files
- Tree-shaking removes unused code automatically

What Vite Optimizes

- Code splitting — large apps load only what's needed
- Asset hashing — cache busting with content-based filenames
- CSS minification and extraction

Preview Before Deploying

- npm run preview — serves the dist/ folder locally to verify before deploying
- Always test the production build — dev mode hides some issues



TypeScript Best Practices

Section 5



Strict Mode and Type Safety Levels

strict: true Enables These Checks

- noImplicitAny — no 'any' type unless you explicitly say so
- strictNullChecks — null and undefined are not assignable to other types
- strictFunctionTypes — stricter checking of function parameter types

Why Strict Mode Matters

- Catches the bugs that matter: null references, implicit any, missing types
- All new projects should start with strict: true
- Legacy JavaScript can be migrated incrementally



Linting with ESLint

Why Lint TypeScript?

- TypeScript catches type errors — ESLint catches code quality issues
- Unused variables, inconsistent style, potential bugs

Setup for Vite Projects

- Vite react-ts template includes ESLint pre-configured
- Uses `@typescript-eslint/parser` and `@typescript-eslint/eslint-plugin`
- `npm run lint` — runs ESLint across your `src/` directory



Common TypeScript Pitfalls

Mistakes to Avoid

- Overusing 'any' — disables type safety entirely
- Forgetting null/undefined handling — leads to runtime crashes
- Confusing type vs interface — use interface for objects, type for unions and tuples
- Not enabling strict mode — misses the most important checks
- Ignoring compiler errors instead of fixing root causes



Compile-Time vs Runtime Types

Types Are Erased at Compile Time

- All type annotations are removed during compilation — JavaScript has no trace of them
- No runtime type checking by default — TypeScript trusts your types at the boundary

When Runtime Validation Is Needed

- API responses — external data may not match your interfaces
- User input — forms and URL parameters are always strings at runtime

Runtime Validation Libraries

- zod, io-ts, valibot — validate data at runtime and infer TypeScript types
- Pair with TypeScript for end-to-end type safety from API to UI



Preparing for React

Section 6



TypeScript Patterns You'll Use in React

Component Props

- Every React component receives props — typed with an interface
- Optional props use ? — children prop is `React.ReactNode`

Generic Hooks

- `useState<number>(0)` — generic tells React what type the state holds
- `useRef<HTMLInputElement>(null)` — generic tells React which DOM element

Event Handlers

- React has its own event types: `React.ChangeEvent<HTMLInputElement>`
- `onClick`, `onChange`, `onSubmit` — all have TypeScript-specific event types



React Type Patterns Preview

```
// Component with typed props
interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: "primary" | "secondary"; // optional union type
}

const Button = ({ label, onClick, variant = "primary" }: ButtonProps) => (
  <button className={variant} onClick={onClick}>{label}</button>
);

// Generic hook usage
const [count, setCount] = useState<number>(0);
const inputRef = useRef<HTMLInputElement>(null);

// Typed event handler
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  setQuery(e.target.value);
};
```

What to Have Ready for React Week

Tooling Checklist

- Node.js installed and working (node --version)
- Can scaffold a Vite + React + TS project from memory
- VS Code with TypeScript and ESLint extensions installed

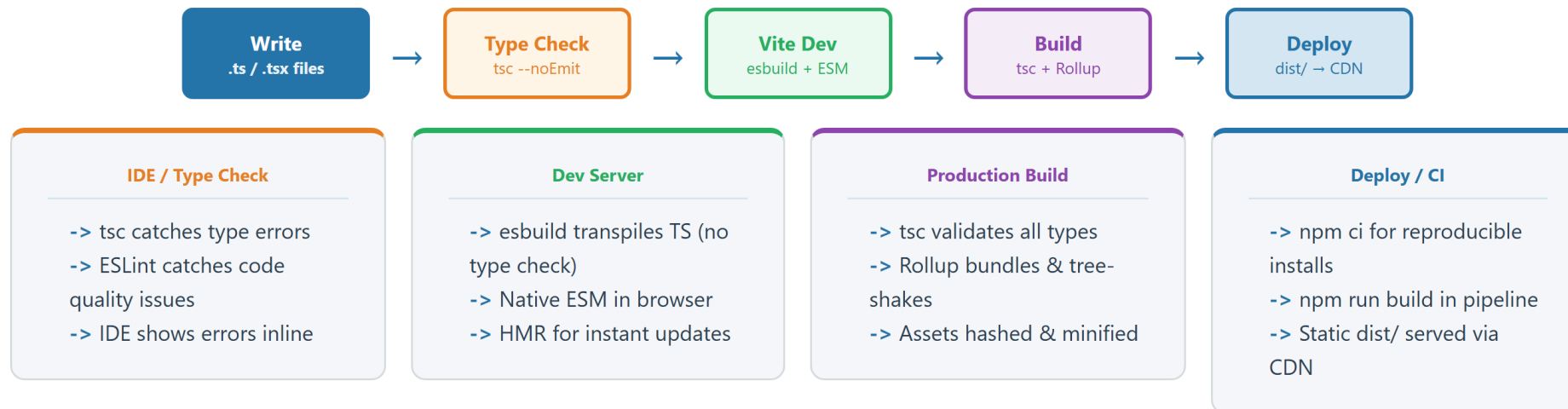
TypeScript Skills Checklist

- Comfortable writing interfaces for object shapes
- Understand generics — at least `function<T>(arg: T): T` pattern
- Can use union types and literal types for constrained values
- Know the difference between type and interface
- Understand import/export and type-only imports



End-to-End Workflow

End-to-End: TypeScript + Vite Workflow



TypeScript types are checked separately from Vite — dev server skips type checking for speed

Key Takeaways — Part 2

Key Takeaways

1. Vite provides instant dev server startup by serving native ES modules — no full bundling in dev
2. Configure Vite in vite.config.ts — plugins, path aliases, server options, and environment variables
3. Use CSS Modules (.module.css) for scoped styles — prevents class name collisions across components
4. Start every project with strict: true — and run tsc separately for type checking (Vite only transpiles)
5. TypeScript types are erased at compile time — validate external data at runtime boundaries
6. Next week: React — you'll use interfaces for props, generics for hooks, and Vite as your build tool

